The Journal of Open Source Software

COMPACT: Comprehensive Prefetching Algorithm Framework

Mahdi Alinejad[1*], Sahand Zoufan[1*], Atiyeh Gheibi-Fetrat[1], Shaahin Hessabi[1], and Hamid Sarbazi-Azad[1,2]

1 Sharif University of Technology, Iran **2** IPM Institute For Research In Fundamental Sciences, Iran ***** These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 27 December 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

COMPACT (Comprehensive Prefetching Algorithm Framework) is a Python-based simulation framework designed to support the rapid development and evaluation of hardware prefetching algorithms. It offers a modular and extensible environment that allows researchers and engineers to explore and test new prefetching techniques without requiring complex hardware changes. The framework includes several common prefetching algorithms, a flexible cache simulator, and built-in tools for analyzing the prefetcher performance. COMPACT is designed for ease of use and customization, making it a practical and valuable resource for the computer architecture research community.

Statement of Need

As the performance gap between processors and main memory continues to grow, hardware prefetching has become a key approach to improving overall system performance. However, developing and evaluating new prefetching algorithms often requires significant time and resources. COMPACT helps address this issue by providing a lightweight and adaptable simulation framework that allows researchers to efficiently test and compare prefetching strategies. By simplifying the evaluation process, COMPACT supports faster experimentation and innovation in the study of hardware prefetching.

Implemented Algorithms

The following prefetching algorithms are implemented in COMPACT. Brief descriptions are provided for each algorithm based on their original publications.

- **Markov Predictor Prefetcher** (Joseph & Grunwald, 1997): The Markov predictor models memory access behavior by representing addresses as states in a Markov chain. Transition probabilities between states are learned from historical access patterns. During execution, the prefetcher predicts the most likely successor addresses based on the current state (accessed address), issuing prefetches for these predicted next addresses.
- **Correlation Prefetcher** (Solihin et al., 2002): The correlation prefetcher uses a helper memory thread that observes sequences of cache misses and learns correlations between them. It records recurring temporal miss patterns and uses these correlations to predict future memory accesses. When the thread detects a similar pattern, it replays the correlated sequence of misses to prefetch data ahead of demand accesses.
- **HDS Prefetcher** (Chilimbi & Hirzel, 2002): The dynamic Hot Data Stream (HDS) prefetcher identifies frequently accessed memory streams by profiling bursty access

patterns. These "hot" streams are modeled using Deterministic Finite State Machines (DFSMs). During profiling, the most frequent sequences of accesses are encoded in DFSMs. At runtime, the prefetcher predicts future accesses by following the DFSM states, prefetching data ahead of demand accesses.

- **TCP Prefetcher** (Hu et al., 2003): The Tag correlating prefetcher operates by correlating cache line tags rather than full memory addresses. It maintains a history of recently accessed cache line tags and predicts the next tag delta based on these correlations. By chaining the predicted tags, TCP can prefetch the next set of cache lines ahead of the demand access. This technique focuses on the correlation of cache line tags, improving prefetch efficiency by predicting multiple cache lines at once.
- **GHB Prefetcher** (Nesbit & Smith, 2004): The global history buffer prefetcher maintains a Global History Buffer (GHB) of recent memory accesses and correlates the address deltas (the differences between consecutive memory addresses) across all Program Counters (PCs). It tracks the history of deltas and uses this information to predict future memory addresses based on recurring delta patterns. These predictions help issue prefetches for future memory accesses before they occur.
- **Store-Ordered Streamer Prefetcher** (Wenisch et al., 2005): The store-ordered streamer prefetcher records store streams generated by producers (e.g., stores to memory). When a consumer (e.g., a load) accesses a line that is part of a producer's store sequence, the prefetcher triggers a series of prefetches for subsequent cache lines in the producer's sequence. This effectively streams data from a producer to a consumer even when their access orders differ.
- **SMS Prefetcher** (Somogyi et al., 2006): The Spatial Memory Streaming (SMS) prefetcher tracks access patterns in memory regions using two main components: an accumulation table that records access history for each region, and a pattern history table that stores spatial signatures. When a memory region is accessed, its pattern signature is matched against stored patterns to prefetch future accesses within that region, improving spatial locality.
- **EBCP Prefetcher** (Chou, 2007): The Epoch-Based Correlation (EBCP) prefetcher partitions the memory access stream into epochs, with boundaries defined by quiescence periods (idle times). It learns correlations between the first miss of an epoch and subsequent misses within that same epoch. Upon detection of a similar epoch trigger, the prefetcher replays the learned miss sequence to prefetch data ahead of demand.
- **Feedback-Directed Prefetcher** (Srinath et al., 2007): The feedback-directed prefetcher monitors feedback metrics such as prefetch accuracy, lateness, and cache pollution. It dynamically adjusts prefetch parameters like prefetch degree based on these metrics over sampling intervals. This tuning process helps balance the trade-off between cache performance improvement and bandwidth usage.
- **Temporal Memory Streaming Prefetcher** (Ferdman et al., 2008): The temporal memory streaming prefetcher records instruction access sequences in a circular miss-order buffer. When an instruction is fetched, it checks the circular buffer for any matching address, and if a match is found, it prefetches subsequent instructions from the recorded sequence. A directory maps instruction addresses to sequence positions, allowing the prefetcher to maintain and replay instruction fetch sequences efficiently.
- **Linear Prefetcher** (Jain & Lin, 2013): The linear prefetcher improves correlated prefetching by transforming irregular memory access patterns through correlation-based address linearization. By mapping related non-contiguous addresses into a logically linear sequence, the prefetcher enables more effective prediction of future accesses without assuming inherent spatial locality.
- **B-Fetch Prefetcher** (Kadjo et al., 2014): The B-fetch prefetcher uses branch prediction

information to guide instruction prefetching in chip multiprocessors. By following predicted branch outcomes and targets, it prefetches instructions along likely future control-flow paths rather than relying only on past fetch history. This approach improves prefetch accuracy and timeliness for workloads with irregular control flow.

- **IMP Prefetcher** (Yu et al., 2015): The indirect memory prefetcher detects indirect memory access patterns where future addresses depend on previous load values (e.g., pointer-based or indexed accesses). By learning the correlation between index values and their corresponding target addresses, the prefetcher predicts future accesses based on the learned index-to-target relationship.
- **Best-Offset Hardware Prefetcher** (Michaud, 2016): The best-offset prefetcher evaluates a predefined set of candidate address offsets and tracks the usefulness of each in generating prefetches. The prefetcher selects the highest-scoring offset based on periodic evaluation and continuously adapts to changes in program access behavior by re-evaluating the offsets.
- **F-TDC Prefetcher** (Jang et al., 2016): The F-TDC prefetcher learns spatial access footprints at the page level using compact footprint vectors. It predicts future memory accesses within a page by proactively fetching cache blocks associated with a previously accessed page, which is particularly useful in tagless DRAM cache designs.
- **Graph Prefetcher** (Ainsworth & Jones, 2016): The graph prefetcher utilizes programmer- or compiler-provided graph traversal semantics to guide prefetching. It identifies memory access patterns corresponding to graph traversal operations and prefetches connected nodes ahead of traversal, optimizing access to irregular graph data structures.
- **TEMPO Prefetcher** (Bhattacharjee, 2017): The TEMPO prefetcher leverages address translation activity as a trigger for prefetching. It combines physical page mappings with cache-line offsets to predict future memory accesses. Non-speculative prefetches are issued as part of the address translation process, allowing better overlap between memory latency and translation.
- **Domino Temporal Data Prefetcher** (Bakhshalipour et al., 2018):The domino prefetcher uses two history tables to capture first-order and second-order correlations between cache misses. Predictions from these tables are chained together, providing more accurate prefetching by capturing deeper temporal relationships. This technique builds on the idea of "domino" effects, where early misses influence later prefetches.
- **Event-Triggered Programmable Prefetcher** (Ainsworth & Jones, 2018): The event-triggered prefetcher is programmable, where prefetching occurs only when specific events (defined by the programmer or system) are triggered based on runtime memory access characteristics (e.g., stride, latency, timing). This allows the prefetching strategy to be adapted dynamically to the workload characteristics.
- **LearnPrefetch Prefetcher** (Hashemi et al., 2018): The Learnprefetch prefetcher uses an LSTM model to learn long-range temporal dependencies from address delta sequences derived from cache misses. By training on historical memory miss sequences, the LSTM predicts future address deltas and generates prefetch candidates, capturing complex temporal access patterns.
- **LearnCluster Prefetcher** (Hashemi et al., 2018): An extension of the Learnprefetch, the Learncluster prefetcher clusters memory accesses using k-means clustering. Each cluster learns its own set of address deltas using the shared LSTM encoder. This improves prediction accuracy by specializing in different access patterns across clusters.
- **Bingo Spatial Data Prefetcher** (Bakhshalipour et al., 2019): The bingo prefetcher tracks memory access patterns within regions using bit-vector footprints. It associates these footprints with specific trigger events. When a trigger matches a stored footprint,

DRAFT

prefetches are issued for all cache blocks within the region indicated by the matched footprint.

- **DSPatch Prefetcher** (Bera et al., 2019): The Dual Spatial Pattern (DSPatch) prefetcher maintains two types of footprints: accuracy-biased and coverage-biased patterns for each spatial signature. Depending on the access behavior, it selects the appropriate pattern to balance accuracy and coverage. Prefetches are issued by aligning the selected footprint with the region's trigger offset.
- **Metadata Prefetcher** (Wu, Nathella, Sunwoo, et al., 2019): The metadata prefetcher maintains per-PC metadata to track correlations between trigger addresses and their predicted target addresses. Confidence counters are associated with each correlation, and prefetches are only issued when these counters exceed a defined threshold, reducing the overhead of ineffective prefetches.
- **Perceptron Prefetcher** (Bhatia et al., 2019): The Perceptron prefetcher uses a Perceptron classifier to filter prefetch candidates by evaluating features extracted from the memory access stream. The Perceptron is trained online using feedback from previous prefetch decisions, reinforcing high-confidence prefetches and suppressing low-confidence ones to reduce ineffective prefetches and cache pollution.
- **Triage Prefetcher** (Wu, Nathella, Pusdesris, et al., 2019): The triage prefetcher operates with on-chip metadata that tracks temporal correlations between consecutive memory addresses. The metadata cache is adaptively sized based on observed prefetch effectiveness, and prefetches are issued when high-confidence temporal correlations are identified.
- **IPCP Prefetcher** (Pakalapati & Panda, 2020): The IPCP prefetcher classifies memory access patterns based on PCs into constant stride, spatial signatures, or complex patterns. It uses specialized sub-prefetchers for each classification, allowing the prefetching strategy to adapt to different types of memory access patterns.
- **Neural Prefetcher** (Shi et al., 2021): The neural prefetcher combines a global LSTM model for capturing long-range memory access patterns with per-PC linear models to learn localized instruction-specific behaviors. This hierarchical structure enables the prefetcher to adapt to both global trends and local, per-instruction access patterns.
- **Triangel Prefetcher** (Ainsworth & Mukhanov, 2024): The triangel prefetcher uses both pattern-based prediction and neighbor-based prediction. It identifies recurring access patterns through training and tracks Markov-style temporal metadata to predict neighboring address relationships. This hybrid approach allows the prefetcher to optimize both accuracy and timeliness in temporal prefetching.

References

- (Joseph & Grunwald, 1997)
- (Solihin et al., 2002)
- (Chilimbi & Hirzel, 2002)
- (Hu et al., 2003)
- (Nesbit & Smith, 2004)
- (Wenisch et al., 2005)
- (Somogyi et al., 2006)
- (Chou, 2007)
- (Srinath et al., 2007)
- (Ferdman et al., 2008)
- (Jain & Lin, 2013)
- (Kadjo et al., 2014)
- (Yu et al., 2015)

DRAFT

- (Michaud, 2016)
- (Jang et al., 2016)
- (Ainsworth & Jones, 2016)
- (Bhattacharjee, 2017)
- (Bakhshalipour et al., 2018)
- (Ainsworth & Jones, 2018)
- (Hashemi et al., 2018)
- (Bakhshalipour et al., 2019)
- (Bera et al., 2019)
- (Wu, Nathella, Sunwoo, et al., 2019)
- (Bhatia et al., 2019)
- (Wu, Nathella, Pusdesris, et al., 2019)
- (Pakalapati & Panda, 2020)
- (Shi et al., 2021)
- (Ainsworth & Mukhanov, 2024)

Ainsworth, S., & Jones, T. M. (2016). Graph prefetching using data structure knowledge. *Proceedings of the 2016 International Conference on Supercomputing*, 1–11. https://doi.org/10.1145/2925426.2926254

Ainsworth, S., & Jones, T. M. (2018). An event-triggered programmable prefetcher for irregular workloads. *ACM Sigplan Notices*, *53*(2), 578–592. https://doi.org/10.1145/3296957.3173189

Ainsworth, S., & Mukhanov, L. (2024). Triangel: A high-performance, accurate, timely on-chip temporal prefetcher. *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 1202–1216. https://doi.org/10.1109/isca59077.2024.00090

Bakhshalipour, M., Lotfi-Kamran, P., & Sarbazi-Azad, H. (2018). Domino temporal data prefetcher. *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 131–142. https://doi.org/10.1109/hpca.2018.00021

Bakhshalipour, M., Shakerinava, M., Lotfi-Kamran, P., & Sarbazi-Azad, H. (2019). Bingo spatial data prefetcher. *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 399–411. https://doi.org/10.1109/hpca.2019.00053

Bera, R., Nori, A. V., Mutlu, O., & Subramoney, S. (2019). Dspatch: Dual spatial pattern prefetcher. *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 531–544.

Bhatia, E., Chacon, G., Pugsley, S., Teran, E., Gratz, P. V., & Jiménez, D. A. (2019). Perceptron-based prefetch filtering. *Proceedings of the 46th International Symposium on Computer Architecture*, 1–13. https://doi.org/10.1145/3307650.3322207

Bhattacharjee, A. (2017). Translation-triggered prefetching. *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, 63–76. https://doi.org/10.1145/3093336.3037705

Chilimbi, T. M., & Hirzel, M. (2002). Dynamic hot data stream prefetching for general-purpose programs. *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation*, 199–209. https://doi.org/10.1145/543552.512554

Chou, Y. (2007). Low-cost epoch-based correlation prefetching for commercial applications. *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*, 301–313. https://doi.org/10.1109/micro.2007.39

Ferdman, M., Wenisch, T. F., Ailamaki, A., Falsafi, B., & Moshovos, A. (2008). Temporal instruction fetch streaming. *2008 41st IEEE/ACM International Symposium on Microarchitecture*, 1–10. https://doi.org/10.1109/micro.2008.4771774

Hashemi, M., Swersky, K., Smith, J., Ayers, G., Litz, H., Chang, J., Kozyrakis, C., &

DRAFT

Ranganathan, P. (2018). Learning memory access patterns. *International Conference on Machine Learning*, 1919–1928.

Hu, Z., Martonosi, M., & Kaxiras, S. (2003). TCP: Tag correlating prefetchers. *The Ninth International Symposium on High-Performance Computer Architecture, 2003. HPCA-9 2003. Proceedings.*, 317–326. https://doi.org/10.1109/hpca.2003.1183549

Jain, A., & Lin, C. (2013). Linearizing irregular memory accesses for improved correlated prefetching. *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture*, 247–259. https://doi.org/10.1145/2540708.2540730

Jang, H., Lee, Y., Kim, J., Kim, Y., Kim, J., Jeong, J., & Lee, J. W. (2016). Efficient footprint caching for tagless dram caches. *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 237–248. https://doi.org/10.1109/hpca.2016.7446068

Joseph, D., & Grunwald, D. (1997). Prefetching using markov predictors. *Proceedings of the 24th Annual International Symposium on Computer Architecture*, 252–263. https://doi.org/10.1109/12.752653

Kadjo, D., Kim, J., Sharma, P., Panda, R., Gratz, P., & Jimenez, D. (2014). B-fetch: Branch prediction directed prefetching for chip-multiprocessors. *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*, 623–634. https://doi.org/10.1109/micro.2014.29

Michaud, P. (2016). Best-offset hardware prefetching. *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 469–480. https://doi.org/10.1109/hpca.2016.7446087

Nesbit, K. J., & Smith, J. E. (2004). Data cache prefetching using a global history buffer. *10th International Symposium on High Performance Computer Architecture (HPCA'04)*, 96–96. https://doi.org/10.1109/mm.2005.6

Pakalapati, S., & Panda, B. (2020). Bouquet of instruction pointers: Instruction pointer classifier-based spatial hardware prefetching. *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 118–131. https://doi.org/10.1109/isca45697.2020.00021

Shi, Z., Jain, A., Swersky, K., Hashemi, M., Ranganathan, P., & Lin, C. (2021). A hierarchical neural model of data prefetching. *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 861–873.

Solihin, Y., Lee, J., & Torrellas, J. (2002). Using a user-level memory thread for correlation prefetching. *ACM SIGARCH Computer Architecture News*, *30*(2), 171–182. https://doi.org/10.1109/isca.2002.1003576

Somogyi, S., Wenisch, T. F., Ailamaki, A., Falsafi, B., & Moshovos, A. (2006). Spatial memory streaming. *ACM SIGARCH Computer Architecture News*, *34*(2), 252–263. https://doi.org/10.1109/isca.2006.38

Srinath, S., Mutlu, O., Kim, H., & Patt, Y. N. (2007). Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers. *2007 IEEE 13th International Symposium on High Performance Computer Architecture*, 63–74. https://doi.org/10.1109/hpca.2007.346185

Wenisch, T. F., Somogyi, S., Hardavellas, N., Kim, J., Gniady, C., Ailamaki, A., & Falsafi, B. (2005). Store-ordered streaming of shared memory. *14th International Conference on Parallel Architectures and Compilation Techniques (PACT'05)*, 75–84. https://doi.org/10.1109/pact.2005.37

Wu, H., Nathella, K., Pusdesris, J., Sunwoo, D., Jain, A., & Lin, C. (2019). Temporal prefetching without the off-chip metadata. *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 996–1008. https://doi.org/10.1145/3352460.3358300

DRAFT

Wu, H., Nathella, K., Sunwoo, D., Jain, A., & Lin, C. (2019). Efficient metadata management for irregular data prefetching. *Proceedings of the 46th International Symposium on Computer Architecture*, 449–461. https://doi.org/10.1145/3307650.3322225

Yu, X., Hughes, C. J., Satish, N., & Devadas, S. (2015). IMP: Indirect memory prefetcher. *Proceedings of the 48th International Symposium on Microarchitecture*, 178–190.

DRAFT